

Documentation:

DMA Porting in Zephyr RTOS

Lakshmeesh K P

Note: This documentation is based on Zephyr RTOS version 4.2.0

Introduction to DMA:

Typically, when we do an ADC read, the data is transferred in two steps. The data is first read from the ADC buffer and written into a CPU register, then again CPU writes that register data back into the required memory location. Doing this continuously is a very inefficient use of CPU, where it could be working on other more harder tasks. we use DMA to make this an efficient single step transfer.

DMA helps offload this redundant work from the CPU. The CPU can command the DMA to transfer data from one location / peripheral to another, and then the CPU can do other useful work, allowing for very high throughput and also parallel execution of work which is helpful for real-time applications.

Portability issue:

While Zephyr does provide a unified API to make code as portable as possible, DMA is deeply tied to the underlying hardware. Thus, as mentioned by Zephyr, it's not completely portable. Meaning it will always require minor architecture-specific adjustments when migrating across different chip vendors.

The implementation of DMA hardware across different manufacturers are very different. They have unique requirements, peripheral interactions, and features. It is not that it is impossible to make it portable (PCs do it), but doing so is complex and adds a lot of code overhead, compared to other devices like ADC. It is a trade-off between portability and the complexity overhead.

Disclaimer:

The way I had made DMA work for me was by hard-coding specific registers within the application code. I didn't have access to a Pico at the time of creating this document, when I was also making efforts to move all hardcoded values into the overlays, to maximise portability. The new codes haven't been tested and may not work; I have written them purely by intuition of how it should work.

I have uploaded the working examples on github for m2m and p2m dma transfers.

These are the pico specific hardcoded values used by me:

- ADC FIFO register address (from rp2040 datasheet) = 0x4004C00C

This is the source address for the DMA to start the ADC transfer from.

- DMA slot for ADC (as per zephyr API) = 0x0B

I had directly hardcoded the dma_slot in the dma config as 0x0b in the initial phase for quick testing.

References:

bindings files: raspberrypi,pico-dma.yaml, base.yaml, zephyr/dt-bindings/dma/rpi_pico_dma.h

dts files: rp2040.dtsi, pico board files

driver files: dma_rpi_pico.c

Implementation:

We see that in any of the .dtsi files, the properties “dmas”, “dma-names” is not specified. So I assume we have to manually specify them in the overlays to connect the DMA channel and DMA request (dreq) signal with the driver.

As per the dma bindings file (raspberrypi,pico-dma.yaml), we are supposed to get the dreq slot from the rpi_pico_dma.h file.

For Pico atleast, there is no restriction on what “dma-names” should be set. But you can find online, some boards require “tx” or “rx” in their bindings files.

```
&adc {
    status = "okay";
    dmas = <&dma 0 RPI_PICO_DMA_SLOT_ADC>;
    // dma-names = "rx";
};
```

Then using Zephyr [DMA macros](#), we can interface with it in the application code.

eg: DT_DMAS_CELL_BY_NAME(DT_NODELABEL(adc), rx, channel)

Zephyr DMA APIs are portable only for memory-to-memory transfer.

Let's say we want to do peripheral-to-memory transfer from an ADC. If you check out the pico DMA driver, nothing is mentioned about configuring an ADC. Ie, we manually have to configure the ADC for our needs. Another thing is that Zephyr ADC apis are not enough for this task either. Thus, the portability breaks, and we have to resort to pico HAL apis to configure the ADC separately.

```
#include "hardware/adc.h"

adc_init();
adc_gpio_init(26); //adc0=gpio26
adc_select_input(0); //channel 0
adc_fifo_setup(true,true, 1,false,false);
adc_set_clkdiv(31.25); // 48000000/(96*16000) //see pg599 of rp2040 datasheet
adc_run(true);
```

To configure the DMA, we need to specify details about the data transfer.

```
//configuring
block_cfg.source_address = 0x4004C00C; //ADC FIFO address
block_cfg.dest_address   = (uint32_t)buffer;
block_cfg.block_size     = sizeof(buffer);
block_cfg.source_addr_adj = DMA_ADDR_ADJ_NO_CHANGE;
block_cfg.dest_addr_adj  = DMA_ADDR_ADJ_INCREMENT;

dma_cfg.channel_direction = PERIPHERAL_TO_MEMORY;
dma_cfg.source_data_size  = 2;
dma_cfg.dest_data_size    = 2;
dma_cfg.block_count       = 1;
dma_cfg.head_block        = &block_cfg;
dma_cfg.dma_slot          = 0x0b; //related to dreq
// dma_cfg.dma_slot       = RPI_PICO_DMA_SLOT_ADC;
dma_cfg.dma_callback      = dma_callback;
```

This example transfers a single “block” of data. The block properties are defined in block_cfg, which tells from where to where data has to be transferred, how large the data is, how to proceed to the next data element.

after successful block transfer, the callback function is called. The callback function is strictly required to be of this given format by zephyr.

```
void dma_callback(const struct device *dev, void *user_data, uint32_t channel, int status){
    if (status < 0) {
        printk("DMA error\n");
        return;
    }
    printk("DMA block complete\n");
}
```

Another example where we control the start and stop of DMA streaming using a shell is also provided on the [github repo](#).